

Digital Solutions 2025

Digital Solutions Team

A textbook aligned with the Digital Solutions 2025 Syllabus

Table of Contents

1 Unit 1: Creating with code	3
1.1 Chapter 1: Understanding Digital Problems	3
1.2 Chapter 2: User Experiences and Interfaces	6
1.3 Chapter 3: Algorithms and Programming Techniques	7
1.4 Chapter 4: Programmed Solutions	8
2 Unit 2: Application and data solutions	11
2.1 Chapter 5: Data-driven Problems and Solution Requirements	11
2.2 Chapter 6: Data and Programming Techniques	12
2.3 Chapter 7: Prototype Data Solutions	12
3 Unit 3: Digital innovation	15
3.1 Chapter 8: Interactions Between Users, Data, and Digital Systems	15
3.2 Chapter 9: Real-world Problems and Solution Requirements	16
3.3 Chapter 10: Innovative Digital Solutions	16
4 Unit 4: Digital impacts	19
4.1 Chapter 11: Digital Methods for Exchanging Data	19
4.2 Chapter 12: Complex Digital Data Exchange Problems	20
4.3 Chapter 13: Prototype Digital Data Exchanges	20
5 Glossary	23
6 Index	24

1 Unit 1: Creating with code

1.1 Chapter 1: Understanding Digital Problems

Aims and Objectives:

- Understand methods of breaking down problems into parts using computational thinking.
- Analyse the personal, social, and economic impacts of digital problems.
- Explore existing and emerging solutions.
- Determine the scope, constraints, and requirements of a problem.

Key Information

Digital problems arise when there is a human need, want, or opportunity that requires a new or re-imagined digital solution. Understanding the problem is the most critical step in software development!

1.1.1 Introduction to Digital Problems

Every piece of software, application, or system was designed to solve a problem.

The computer was born to solve problems that did not exist before.

— Bill Gates

Whether it is a simple mobile app to track your daily steps or a complex machine learning algorithm for medical diagnosis, understanding the core problem is paramount.

Historical Context

The first computer bug was an actual bug! In 1947, Grace Hopper and her team found a moth trapped in a relay of the Harvard Mark II computer. They taped it into the logbook, noting it as the first actual case of “bug” being found.

Source: General History

Women in Tech

Grace Hopper was an American computer scientist and US Navy rear admiral. She was one of the first programmers of the Harvard Mark I computer and a pioneer of computer programming who invented one of the first linkers. She popularised the idea of machine-independent programming languages, which led to the development of COBOL.

1.1.2 Breaking Down Problems

Computational thinking involves breaking down a large, complex problem into smaller, more manageable parts. This process is called **decomposition**.

Let's look at an example. If we want to build a system to calculate student grades, we might break it down into:

1. Input: Reading student marks.
2. Process: Calculating the average and determining the grade.
3. Output: Displaying the results.

Pseudocode

```
// Worked Example 1:
Grade Calculation
BEGIN
  READ marks[]
  sum = 0
  FOR EACH mark IN marks:
    sum = sum + mark
  ENDFOR
  average = sum /
LENGTH(marks)
  IF average >= 50 THEN
    PRINT 'Pass'
  ELSE
    PRINT 'Fail'
  ENDIF
END
```

JavaScript

```
// Worked Example 1:
Grade Calculation
function
calculateGrade(marks) {
  let sum = 0;
  for (let mark of marks)
  {
    sum += mark;
  }
  let average = sum /
marks.length;
  if (average >= 50) {
    console.log('Pass');
  } else {
    console.log('Fail');
  }
}
```

Python

```
# Worked Example 1: Grade
Calculation
def
calculate_grade(marks):
    total_sum = sum(marks)
    average = total_sum /
len(marks)

    if average >= 50:
        print('Pass')
    else:
        print('Fail')
```

Pseudocode

```
// Worked Example 2:
Checking Even or Odd
BEGIN
  READ number
  IF number MOD 2 == 0
THEN
    PRINT 'Even'
  ELSE
    PRINT 'Odd'
```

JavaScript

```
// Worked Example 2:
Checking Even or Odd
function
checkEvenOdd(number) {
  if (number % 2 === 0) {
    console.log('Even');
  } else {
    console.log('Odd');
  }
}
```

Python

```
# Worked Example 2:
Checking Even or Odd
def
check_even_odd(number):
    if number % 2 == 0:
        print('Even')
    else:
        print('Odd')
```



```
ENDIF
END
```

Pseudocode

```
// Scaffolded Question 1:
Find Maximum
// Complete the loop to
find the max value.
BEGIN
  READ values[]
  max = values[0]
  FOR i = 1 TO
    LENGTH(values) - 1
    IF _____ THEN
      _____
    ENDIF
  ENDFOR
  PRINT max
END
```

JavaScript

```
// Scaffolded Question 1:
Find Maximum
// Complete the loop to
find the max value.
function findMax(values)
{
  let max = values[0];
  for (let i = 1; i <
    values.length; i++) {
    if (_____) {
      _____
    }
  }
  console.log(max);
}
```

Python

```
# Scaffolded Question 1:
Find Maximum
# Complete the loop to
find the max value.
def find_max(values):
    max_val = values[0]
    for i in range(1,
    len(values)):
        if _____:
            _____
    print(max_val)
```

Pseudocode

```
// Scaffolded Question 2:
Sum of Evens
// Complete the logic to
sum only even numbers.
BEGIN
  READ numbers[]
  sum = 0
  FOR EACH num IN numbers:
    IF _____ THEN
      sum = _____
    ENDIF
  ENDFOR
  PRINT sum
END
```

JavaScript

```
// Scaffolded Question 2:
Sum of Evens
// Complete the logic to
sum only even numbers.
function
sumEvens(numbers) {
  let sum = 0;
  for (let num of numbers)
  {
    if (_____) {
      sum = _____
    }
  }
  console.log(sum);
}
```

Python

```
# Scaffolded Question 2:
Sum of Evens
# Complete the logic to
sum only even numbers.
def sum_evens(numbers):
    total = 0
    for num in numbers:
        if _____:
            total = _____
    print(total)
```

Pseudocode

```
// Unscaffolded Question
1: Count Vowels
// Write an algorithm
to count the number of
vowels in a string.
```

JavaScript

```
// Unscaffolded Question
1: Count Vowels
// Write a function
to count the number of
vowels in a string.
```

Python

```
# Unscaffolded Question
1: Count Vowels
# Write a function to
count the number of
vowels in a string.
```

Pseudocode

JavaScript

Python

```
// Unscaffolded Question
2: Reverse Array
// Write an algorithm to
reverse an array without
using built-in methods.
```

```
// Unscaffolded Question
2: Reverse Array
// Write a function to
reverse an array without
using built-in methods.
```

```
# Unscaffolded Question
2: Reverse Array
# Write a function to
reverse a list without
using built-in methods.
```

1.1.3 Revision Questions

1. Define computational thinking and explain the process of decomposition.
2. Provide an example of a digital problem in your local community and discuss its social impact.
3. What is the significance of understanding the scope and constraints before writing code?

—

1.2 Chapter 2: User Experiences and Interfaces

Aims and Objectives:

- Recognise the meaning and importance of user experience (UX).
- Understand usability principles including accessibility, effectiveness, safety, utility, and learnability.
- Explore the elements and principles of visual communication.

Key Information

User Experience (UX) encompasses all aspects of the end-user's interaction with the company, its services, and its products. Good UX is invisible, while bad UX is immediately noticeable!

1.2.1 Introduction to UX

The user interface is the bridge between the human and the digital system. Designing effective interfaces requires careful consideration of the user's needs, capabilities, and the environment in which they operate.

<u>DESIGNER</u>	<u>WHAT THEY ARE RESPONSIBLE FOR</u>
UI	ELEMENTS OF THE INTERFACE THAT THE USER ENCOUNTERS
UX	THE USER'S EXPERIENCE OF USING THE INTERFACE TO ACHIEVE GOALS
UZ	THE PSYCHOLOGICAL ROOTS OF THE USER'S MOTIVATION FOR SEEKING OUT THE INTERACTION
U α	THE USER'S SELF-ACTUALIZATION
U Ω	THE ARC OF THE USER'S LIFE
U ∞	LIFE'S EXPERIENCE OF TIME
U●	THE ARC OF THE MORAL UNIVERSE

Figure 1: A humorous take on UI vs UX.¹

For further insights on creating great user experiences, watch this video on the principles of UX design:
<https://www.youtube.com/watch?v=O94kYyzqclQ>

1.2.2 Revision Questions

1. Explain the difference between UI and UX.
2. List and define three useability principles.
3. How do visual communication principles enhance the learnability of an application?

—

1.3 Chapter 3: Algorithms and Programming Techniques

Aims and Objectives:

- Recognise and describe programming syntax and rules.

¹Source: xkcd.com

- Understand that simple algorithms consist of input, process, and output.
- Develop algorithms using pseudocode.

1.3.1 Introduction to Algorithms

An algorithm is simply a step-by-step set of instructions used to solve a specific problem. Before we write code in a specific language, we often design the logic using pseudocode.

Historical Context

Ada Lovelace is often considered the first computer programmer. In the 1840s, she wrote an algorithm intended to be executed by Charles Babbage's early mechanical general-purpose computer, the Analytical Engine.

Source: General History

Women in Tech

Margaret Hamilton was the Director of the Software Engineering Division of the MIT Instrumentation Laboratory, which developed on-board flight software for NASA's Apollo program. She coined the term "software engineering".

1.3.2 Revision Questions

1. What are the three fundamental stages of a simple algorithm?
2. Why is pseudocode useful when designing digital solutions?
3. Name the five basic features of programming.

—

1.4 Chapter 4: Programmed Solutions

Aims and Objectives:

- Use a text-based language to apply operators and logic.
- Generate reusable coded components.
- Evaluate and refine programmed solutions.

1.4.1 Bringing It All Together

Once the logic is defined and the interface is designed, the final step in Unit 1 is implementing the programmed solution. This involves writing syntax-correct code, debugging, and testing.

Pseudocode

```
// Worked Example 1:
Calculate Area
BEGIN
  READ length
  READ width
  area = length * width
  PRINT area
END
```

JavaScript

```
// Worked Example 1:
Calculate Area
function
calculateArea(length,
width) {
  let area = length *
width;
  console.log(area);
}
```

Python

```
# Worked Example 1:
Calculate Area
def
calculate_area(length,
width):
  area = length * width
  print(area)
```

Pseudocode

```
// Worked Example 2:
Check Temperature
BEGIN
  READ temp
  IF temp > 30 THEN
    PRINT 'Hot'
  ELSE
    PRINT 'Cold'
  ENDIF
END
```

JavaScript

```
// Worked Example 2:
Check Temperature
function checkTemp(temp)
{
  if (temp > 30) {
    console.log('Hot');
  } else {
    console.log('Cold');
  }
}
```

Python

```
# Worked Example 2: Check
Temperature
def check_temp(temp):
  if temp > 30:
    print('Hot')
  else:
    print('Cold')
```

Pseudocode

```
// Scaffolded Question 1:
Check Age
BEGIN
  READ age
  IF age >= _____ THEN
    PRINT 'Adult'
  ELSE
    PRINT '_____'
  ENDIF
END
```

JavaScript

```
// Scaffolded Question 1:
Check Age
function checkAge(age) {
  if (age >= _____) {
    console.log('Adult');
  } else {
    console.log('_____');
  }
}
```

Python

```
# Scaffolded Question 1:
Check Age
def check_age(age):
  if age >= _____:
    print('Adult')
  else:
    print('_____')
```

Pseudocode

```
// Scaffolded Question 2:
Calculate Discount
BEGIN
  READ price
  discount = price *
_____
  final_price = price -
discount
  PRINT _____
END
```

JavaScript

```
// Scaffolded Question 2:
Calculate Discount
function
calculateDiscount(price)
{
  let discount = price *
_____;
  let finalPrice = price
- discount;
```

Python

```
# Scaffolded Question 2:
Calculate Discount
def calculate_discount(price):
  discount = price *
_____
  final_price = price -
discount
  print(_____)
```

```
console.log(____);
}
```

Pseudocode

```
// Unscaffolded Question
1: Factorial
// Write an algorithm to
compute the factorial of
a number.
```

JavaScript

```
// Unscaffolded Question
1: Factorial
// Write a JavaScript
function for factorial.
```

Python

```
# Unscaffolded Question
1: Factorial
# Write a Python function
for factorial.
```

Pseudocode

```
// Unscaffolded Question
2: Fibonacci
// Write an algorithm
to print the first N
Fibonacci numbers.
```

JavaScript

```
// Unscaffolded Question
2: Fibonacci
// Write a JavaScript
function for Fibonacci.
```

Python

```
# Unscaffolded Question
2: Fibonacci
# Write a Python function
for Fibonacci.
```

1.4.2 Revision Questions

1. Differentiate between syntax errors and logic errors.
2. Explain the scope of a variable and the difference between local and global scope.
3. Why are consistent naming conventions important in programming?

2 Unit 2: Application and data solutions

2.1 Chapter 5: Data-driven Problems and Solution Requirements

Aims and Objectives:

- Understand the nature of data-driven problems.
- Recognise data types, constraints, and primary and foreign keys.
- Explore the social and economic impacts of data storage.

Key Information

Data is raw, unorganised facts. Information is data that has been processed, organised, structured, or presented in a given context to make it useful. Wisdom is the application of this information to make decisions.

2.1.1 The Nature of Data-Driven Problems

In the modern world, data is being generated at an unprecedented rate. Managing this data—inserting, validating, structuring, and retrieving it—is the core of data-driven digital solutions.

Historical Context

In 1970, Edgar F. Codd, a computer scientist at IBM, published a paper titled “A Relational Model of Data for Large Shared Data Banks”. This paper laid the foundation for relational databases, which are still the dominant method of storing structured data today!

Source: General History

2.1.2 Modeling Data

Before writing any code or SQL queries, it is essential to model the data and its flow through a system. This is done using context diagrams and Data Flow Diagrams (DFDs).

2.1.3 Revision Questions

1. Differentiate between data, information, and wisdom.

2. What are the potential privacy impacts of storing user data in a cloud database?
3. Define the purpose of a primary key in a relational database.

—

2.2 Chapter 6: Data and Programming Techniques

Aims and Objectives:

- Understand relational database structures and normalisation.
- Differentiate between data validation and verification.
- Develop algorithms for data processing.

2.2.1 Database Structures and Normalisation

Normalisation is the process of organising data in a database to reduce redundancy and improve data integrity. The syllabus requires understanding up to Third Normal Form (3NF).

Women in Tech

Dr. Karen Spärck Jones was a pioneering British computer scientist whose work in the 1970s introduced the concept of Inverse Document Frequency (IDF). Her work is foundational to the search engines we use every day to query massive databases!

2.2.2 Revision Questions

1. Explain the difference between data validation and data verification.
2. Describe the conditions required for a database table to be in Third Normal Form (3NF).
3. Why is reducing data redundancy important?

—

2.3 Chapter 7: Prototype Data Solutions

Aims and Objectives:

- Use SQL to retrieve and modify data.
- Generate and test prototype data solutions.

2.3.1 Introduction to SQL

Structured Query Language (SQL) is the standard language for dealing with Relational Databases.

Figure 2: Always sanitize your database inputs!²**Pseudocode**

```
// Worked Example 1:
Retrieve Users
// Retrieve all active
users.
BEGIN
  QUERY database:
    SELECT first_name,
last_name
  FROM users
  WHERE status = 'active'
END
```

JavaScript

```
// Worked Example 1:
Retrieve Users
const sql = `
  SELECT first_name,
last_name
  FROM users
  WHERE status = 'active'
`;
db.query(sql, (err,
results) => {
  console.log(results);
});
```

Python

```
# Worked Example 1:
Retrieve Users
sql = '''
  SELECT first_name,
last_name
  FROM users
  WHERE status = 'active'
'''
cursor.execute(sql)
results =
cursor.fetchall()
print(results)
```

Pseudocode

```
// Worked Example 2:
Update Record
// Update user email.
BEGIN
  QUERY database:
    UPDATE users
      SET email =
'new@email.com'
    WHERE user_id = 5
END
```

JavaScript

```
// Worked Example 2:
Update Record
const sql = `
  UPDATE users
    SET email =
'new@email.com'
  WHERE user_id = 5
`;
db.query(sql, (err,
result) => {
  console.log('Record
updated');
});
```

Python

```
# Worked Example 2:
Update Record
sql = '''
  UPDATE users
    SET email =
'new@email.com'
  WHERE user_id = 5
'''
cursor.execute(sql)
db.commit()
print('Record updated')
```

Pseudocode

```
// Scaffolded Question 1:
Insert Data
// Complete the query to
add a new product.
```

JavaScript

```
// Scaffolded Question 1:
Insert Data
const sql = `
  INSERT INTO products
```

Python

```
# Scaffolded Question 1:
Insert Data
sql = '''
  INSERT INTO products
```

²Source: xkcd.com/327

```
BEGIN
  QUERY database:
    INSERT INTO products
      (name, price)
      VALUES (_____, _____)
END
```

```
(name, price)
VALUES (_____, _____)
`;
```

```
(name, price)
VALUES (_____, _____)
'''
```

Pseudocode

```
// Scaffolded Question 2:
Delete Data
// Complete the query to
delete old records.
BEGIN
  QUERY database:
    DELETE FROM logs
    WHERE date < _____
END
```

JavaScript

```
// Scaffolded Question 2:
Delete Data
const sql = `
  DELETE FROM logs
  WHERE date < _____
`;
```

Python

```
# Scaffolded Question 2:
Delete Data
sql = '''
  DELETE FROM logs
  WHERE date < _____
'''
```

Pseudocode

```
// Unscaffolded Question
1: Count Records
// Write a query to count
the number of orders
placed by customer ID 10.
```

JavaScript

```
// Unscaffolded Question
1: Count Records
// Write a JavaScript
function to execute this
query.
```

Python

```
# Unscaffolded Question
1: Count Records
# Write a Python function
to execute this query.
```

Pseudocode

```
// Unscaffolded Question
2: Join Tables
// Write a query to
select the order ID
and customer name by
joining the 'orders' and
'customers' tables.
```

JavaScript

```
// Unscaffolded Question
2: Join Tables
// Write a JavaScript
function to execute this
query.
```

Python

```
# Unscaffolded Question
2: Join Tables
# Write a Python function
to execute this query.
```

2.3.2 Revision Questions

1. Write an SQL statement to create a table named Students with columns for ID, Name, and Grade.
2. What is the purpose of the GROUP BY clause in SQL? Provide an example.
3. How does SQL injection occur, and how can a developer prevent it in their application code?

3 Unit 3: Digital innovation

3.1 Chapter 8: Interactions Between Users, Data, and Digital Systems

Aims and Objectives:

- Explore the meaning and importance of innovation.
- Recognise components of web, mobile, interactive, and intelligent systems.
- Understand advanced data processes.

Key Information

Innovation isn't just about inventing something entirely new; it often involves applying existing technologies in novel ways to solve problems more efficiently or open up new opportunities.

3.1.1 The Impact of Innovation

Emerging technologies like machine learning, neural networks, and natural language processing are driving a new wave of digital innovation. They transform how systems interact with data and users.

Historical Context

The concept of artificial intelligence dates back to antiquity, but the field was formally founded at a conference at Dartmouth College in 1956. Early pioneers believed that a machine as intelligent as a human being would exist within a generation!

Source: General History

3.1.2 Revision Questions

1. Define 'innovation' in the context of digital solutions.
2. List three components of a web application architecture.
3. How might natural language processing improve human-computer interaction?

—

3.2 Chapter 9: Real-world Problems and Solution Requirements

Aims and Objectives:

- Explore project management tools.
- Determine specific requirements and success criteria for digital solutions.
- Use modular programming.

Women in Tech

Radia Perlman is an American software designer and network engineer. She is most famous for her invention of the Spanning Tree Protocol (STP), which is fundamental to the operation of network bridges and keeps the internet running smoothly!

3.2.1 Revision Questions

1. Why are project management tools essential for software development?
2. Explain the concept of modularity in programming.
3. What is the difference between an input constraint and an environmental limitation?

—

3.3 Chapter 10: Innovative Digital Solutions

Aims and Objectives:

- Generate user-interface prototypes.
- Develop conceptual models and generate program modules.
- Evaluate prototype solutions against success criteria.

3.3.1 Evaluating Prototypes

Evaluation is not just checking if the code runs. It involves heuristic reviews of the interface against useability principles and desk-checking algorithmic steps.

Pseudocode

```
// Worked Example 1: Read
File
// Read data from a text
file.
BEGIN
    OPEN  'data.txt' FOR
READING
```

JavaScript

```
// Worked Example 1: Read
File
const fs = require('fs');
fs.readFile('data.txt',
'utf8', (err, data) => {
    if (err) throw err;
```

Python

```
# Worked Example 1: Read
File
with open('data.txt',
'r') as file:
    for line in file:
        print(line.strip())
```

```

WHILE NOT END_OF_FILE
  READ line
  PRINT line
ENDWHILE
CLOSE 'data.txt'
END

```

```

console.log(data);
});

```

Pseudocode

```

// Worked Example 2:
Write File
// Write data to a text
file.
BEGIN
  OPEN 'output.txt' FOR
  WRITING
    WRITE 'Hello
  Innovation' TO FILE
  CLOSE 'output.txt'
END

```

JavaScript

```

// Worked Example 2:
Write File
const fs = require('fs');
fs.writeFile('output.txt',
'Hello Innovation', err
=> {
  if (err) throw err;
  console.log('Saved!');
});

```

Python

```

# Worked Example 2: Write
File
with open('output.txt',
'w') as file:
  file.write('Hello
  Innovation')
  print('Saved!')

```

Pseudocode

```

// Scaffolded Question 1:
Append to File
// Complete the code
to append data without
overwriting.
BEGIN
  OPEN 'log.txt' FOR
  _____
  WRITE 'New Entry' TO
  FILE
  CLOSE 'log.txt'
END

```

JavaScript

```

// Scaffolded Question 1:
Append to File
const fs = require('fs');
fs.____('log.txt', 'New
Entry', err => {
  // handle error
});

```

Python

```

# Scaffolded Question 1:
Append to File
with open('log.txt',
'____') as file:
  file.write('New
  Entry')

```

Pseudocode

```

// Scaffolded Question 2:
Simple Input Validation
// Ensure user age is
above 0.
BEGIN
  READ age
  IF age _____ 0 THEN
    PRINT 'Valid'
  ELSE
    PRINT '_____'
  ENDIF
END

```

JavaScript

```

// Scaffolded Question 2:
Simple Input Validation
function
validateAge(age) {
  if (age _____ 0) {
    return 'Valid';
  } else {
    return '_____';
  }
}

```

Python

```

# Scaffolded Question 2:
Simple Input Validation
def validate_age(age):
  if age _____ 0:
    return 'Valid'
  else:
    return '_____'

```

Pseudocode

```
// Unscaffolded Question
1: Read CSV File
// Write an algorithm to
read a CSV file line by
line and split data by
commas.
```

JavaScript

```
// Unscaffolded Question
1: Read CSV File
// Write a JavaScript
function to read a CSV
file.
```

Python

```
# Unscaffolded Question
1: Read CSV File
# Write a Python function
to read a CSV file using
the csv module.
```

Pseudocode

```
// Unscaffolded Question
2: Search Array
// Write an algorithm
that searches for a
specific string in an
array and returns its
index.
```

JavaScript

```
// Unscaffolded Question
2: Search Array
// Write a JavaScript
function to perform a
linear search.
```

Python

```
# Unscaffolded Question
2: Search Array
# Write a Python function
to perform a linear
search.
```

3.3.2 Revision Questions

1. Describe two methods for evaluating a user interface prototype.
2. Explain what a 'system boundary' is in a conceptual model.
3. Why is it important to close a file after reading or writing to it in a program?

4 Unit 4: Digital impacts

4.1 Chapter 11: Digital Methods for Exchanging Data

Aims and Objectives:

- Recognise encryption and authentication strategies.
- Understand network performance metrics and transmission principles.
- Use methods like REST, JSON, and XML for data exchange.

Key Information

When transferring data over a network, security is paramount. Encryption ensures confidentiality, hashing ensures integrity, and authentication ensures identity.

4.1.1 Encryption Algorithms

Symmetric encryption uses the same key to encrypt and decrypt data (e.g., AES). Asymmetric encryption uses a public key to encrypt and a private key to decrypt (e.g., RSA).

Historical Context

The Caesar cipher, named after Julius Caesar, is one of the simplest and most widely known encryption techniques. It is a substitution cipher where each letter in the plaintext is replaced by a letter some fixed number of positions down the alphabet.

Source: General History

4.1.2 Revision Questions

1. Differentiate between symmetric and asymmetric encryption.
2. Explain how a hashing algorithm ensures data integrity.
3. Compare JSON and XML as formats for data exchange.

—

4.2 Chapter 12: Complex Digital Data Exchange Problems

Aims and Objectives:

- Analyse factors and risks affecting data security and privacy.
- Develop data flow diagrams for complex exchanges.
- Evaluate the impact of emerging technologies in data exchange.

Women in Tech

Joan Clarke was an English cryptanalyst and numismatist best known for her work as a code-breaker at Bletchley Park during the Second World War. Her work on the Enigma project that decrypted Nazi Germany's secret communications earned her awards and citations.

4.2.1 Revision Questions

1. What are the Australian Privacy Principles, and how do they impact software developers?
2. Define latency and jitter in the context of network performance.
3. How can machine learning be used to detect anomalies in data exchange networks?

—

4.3 Chapter 13: Prototype Digital Data Exchanges

Aims and Objectives:

- Use a suitable programming environment to implement data exchange.
- Manipulate JSON/XML data.
- Evaluate solutions against success criteria.

Pseudocode

```
// Worked Example 1:
Parse JSON
// Extract 'name' from a
JSON string.
BEGIN
    json_string =
'{'  "name":  "Alice",
"age": 18 }'
    data =
PARSE_JSON(json_string)
    PRINT data.name
END
```

JavaScript

```
// Worked Example 1:
Parse JSON
const jsonString =
'{'  "name":  "Alice",
"age": 18 }';
const data =
JSON.parse(jsonString);
console.log(data.name);
```

Python

```
# Worked Example 1: Parse
JSON
import json
json_string = '{ "name":
"Alice", "age": 18 }'
data =
json.loads(json_string)
print(data['name'])
```


Pseudocode

```
// Worked Example 2:
Caesar Cipher Encrypt
// Shift letters by 3.
BEGIN
  FUNCTION encrypt(text,
  shift)
    result = ""
    FOR EACH char IN text:
      new_char =
      SHIFT_CHAR(char, shift)
      result = result +
      new_char
    ENDFOR
    RETURN result
  ENDFUNCTION
END
```

JavaScript

```
// Worked Example 2:
Caesar Cipher Encrypt
function encrypt(text,
  shift) {
  let result = "";
  for (let i = 0; i <
  text.length; i++) {
    let charCode =
    text.charCodeAt(i);
    // Simplified: assumes
    all uppercase letters
    result += String.fromCharCode(
    - 65 + shift) % 26 + 65);
  }
  return result;
}
```

Python

```
# Worked Example 2:
Caesar Cipher Encrypt
def encrypt(text,
  shift):
  result = ""
  for char in text:
    # Simplified:
    assumes all uppercase
    letters
    result +=
    chr((ord(char) - 65 +
    shift) % 26 + 65)
  return result
```

Pseudocode

```
// Scaffolded Question 1:
Stringify JSON
// Convert an object to a
JSON string.
BEGIN
  student = CREATE_OBJECT(name
  grade="A")
  json_string = _____
  (student)
  PRINT json_string
END
```

JavaScript

```
// Scaffolded Question 1:
Stringify JSON
const student = { name:
  "Bob", grade: "A" };
const jsonString =
  JSON._____(student);
console.log(jsonString);
```

Python

```
# Scaffolded Question 1:
Stringify JSON
import json
student = { "name":
  "Bob", "grade": "A" }
json_string =
  json._____(student)
print(json_string)
```

Pseudocode

```
// Scaffolded Question 2:
Simple API Fetch
// Fetch data from a REST
API endpoint.
BEGIN
  url = "https://api.
  example.com/data"
  response = _____ GET
  url
  PRINT response.data
END
```

JavaScript

```
// Scaffolded Question 2:
Simple API Fetch
const url = "https://api.
  example.com/data";
fetch(url)
  .then(response =>
  response.json())
  .then(data =>
  console.log(_____));
```

Python

```
# Scaffolded Question 2:
Simple API Fetch
import requests
url = "https://api.
  example.com/data"
response = _____
  requests._____(url)
print(response._____)()
```

Pseudocode**JavaScript****Python**

```
// Unscaffolded Question
1: Fetch and Parse XML
// Write an algorithm to
fetch XML from a URL and
extract a specific node.
```

```
// Unscaffolded Question
1: Fetch and Parse XML
// Write a JavaScript
function to fetch and
parse XML.
```

```
# Unscaffolded Question
1: Fetch and Parse XML
# Write a
Python function using
xml.etree.ElementTree to
parse fetched XML.
```

Pseudocode

```
// Unscaffolded Question
2: Vigenère Cipher
// Write an algorithm to
encrypt a string using
the Vigenère cipher.
```

JavaScript

```
// Unscaffolded Question
2: Vigenère Cipher
// Write a JavaScript
function to perform
Vigenère encryption.
```

Python

```
# Unscaffolded Question
2: Vigenère Cipher
# Write a Python function
to perform Vigenère
encryption.
```

4.3.1 Revision Questions

1. Why is data compression important when transmitting data over a network?
2. Describe the role of an API in data exchange.
3. Discuss the economic impacts of a major data breach on a business.

5 Glossary

Algorithm: A step-by-step set of operations to be performed to solve a specific problem.

Asymmetric Encryption: A type of encryption that uses two different but mathematically related keys: a public key and a private key.

Computational Thinking: The thought processes involved in formulating a problem and expressing its solution(s) in such a way that a computer can effectively carry out.

Data Flow Diagram (DFD): A visual representation of the flow of data through an information system.

Decomposition: Breaking down a complex problem or system into smaller, more manageable parts.

Foreign Key: A column or group of columns in a relational database table that provides a link between data in two tables.

Innovation: The practical implementation of ideas that result in the introduction of new goods or services or improvement in offering goods or services.

JSON (JavaScript Object Notation): A lightweight data-interchange format that is easy for humans to read and write and easy for machines to parse and generate.

Normalisation: The process of structuring a relational database to reduce data redundancy and improve data integrity.

Primary Key: A specific choice of a minimal set of attributes (columns) that uniquely specify a tuple (row) in a relation (table).

Pseudocode: A plain language description of the steps in an algorithm.

SQL (Structured Query Language): A domain-specific language used in programming and designed for managing data held in a relational database management system.

Symmetric Encryption: A type of encryption where the same key is used to encrypt and decrypt the message.

User Experience (UX): A person's emotions and attitudes about using a particular product, system or service.

XML (eXtensible Markup Language): A markup language that defines a set of rules for encoding documents in a format that is both human-readable and machine-readable.

6 Index

A

- Algorithm
- API
- Asymmetric Encryption
- Authentication

C

- Caesar Cipher
- Computational Thinking

D

- Data Flow Diagram (DFD)
- Database
- Decomposition

E

- Encryption
- Error Handling

F

- Foreign Key

I

- Innovation

J

- JSON

N

- Network Protocols
- Normalisation

P

- Primary Key
- Pseudocode

R

- Relational Database
- REST API

S

- SQL
- Symmetric Encryption

U

- User Experience (UX)
- User Interface (UI)

V

- Validation
- Vigenère Cipher

X

- XML